CST1510 Programming for data communication and networks

Week 8 Extending the Shopping System Project with Client-Server Communication

LAB ACTIVITY

Lab Activity – Implementing Client-Server Communication for Your Project

In this lab, you will extend your chosen project—either the **Smart Vending Machine** or the **Crypto Investment Platform**—by implementing client-server communication. This activity will help you understand how to separate the client interface from the server's data management and transaction processing, preparing you for future enhancements like database storage and a graphical interface.

Lab Objective

Transform your project into a networked application by implementing client-server communication. The client will handle user interactions (e.g., viewing products, adding to cart, or checking portfolio), while the server manages product inventory, transactions, or asset data.

Here is a set of structured guidelines to guide you through implementing a full client-server version of your project. These steps will help you to build a robust, networked shopping system (or apply to a similar project, such as a vending machine or investment platform).

Client-Server Shopping System Implementation Guide

Project Overview

1. **Client Responsibilities**:
   - The client will handle all user interactions, including viewing products, adding items to the cart, and checking out.
   - The client will communicate with the server to request information, process transactions, and receive updates.
2. **Server Responsibilities**:
   - The server will manage product inventory, process orders, and log transactions.
   - The server will respond to client requests, provide data as needed, and update inventory or account balances accordingly.

Step-by-Step Guide

Step 1: Prepare Initial Files and Data

1. **Set Up Inventory File** (`inventory.txt`):
   - Create a `.txt` file to store product information, such as:
     ```plaintext
     ID, Product_Name, Price, Stock
     1, Sneakers, 49.99, 10
     2, Backpack, 29.99, 15
     3, Water Bottle, 9.99, 20
     ```
2. **Create Transaction Log** (`transactions.txt`):
   - This file will log each transaction processed by the server, helping to track purchases and inventory updates.

Step 2: Server-Side Implementation

A. Server Setup

1. **Initialize Server Socket**:

- Set up the server to listen on a specific port, using the `socket` library.
- Allow multiple client connections if required by the project scope.

2. **Define Server Functions**:

   - **`load_inventory()`**: Reads `inventory.txt` and loads data into a dictionary structure.
   - **`save_inventory()`**: Updates `inventory.txt` when items are purchased.
   - **`log_transaction(cart)`**: Logs purchases to `transactions.txt`.

B. Server Command Functions

1. **VIEW**:

   - Returns a list of products, including IDs, names, prices, and stock quantities, to the client.

2. **ADD**:

   - Checks if a product is in stock and adds it to the client's cart.
   - Updates inventory to reflect changes.

3. **CART**:

   - Sends the current cart contents to the client, including total cost.

4. **CHECKOUT**:

   - Confirms all items in the cart, processes payments (or deducts funds), and finalizes the order.
   - Updates inventory and logs the transaction.

C. Example Server Code Structure

Here's a basic layout of server-side functions. Expand as needed for your specific project:

```
import socket

def load_inventory():
    inventory = {}
    # Load from inventory.txt
    return inventory
```

```python
def save_inventory(inventory):
    # Save updated inventory to file
    pass

def log_transaction(cart):
    # Append transaction details to transactions.txt
    pass

def handle_client(client_socket):
    while True:
        request = client_socket.recv(1024).decode()
        # Process request based on command type
        if request.startswith("VIEW"):
            # Send inventory data
            pass
        elif request.startswith("ADD"):
            # Add item to cart
            pass
        elif request.startswith("CART"):
            # Send cart details
            pass
        elif request.startswith("CHECKOUT"):
            # Process checkout
            pass
        else:
            break

# Server setup
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('localhost', 5000))
server_socket.listen(5)

while True:
    client_socket, _ = server_socket.accept()
    handle_client(client_socket)
```

Step 3: Client-Side Implementation

A. Client Setup

1. **Initialize Client Socket**:
   - Set up the client to connect to the server on the specified IP and port.
2. **Define Client Functions**:
   - Implement functions for viewing products, adding items to the cart, and checking out.

B. Client Command Functions

1. **VIEW**:
   - Sends a `VIEW` request to the server and displays the available products.
2. **ADD**:
   - Prompts the user for a product ID and quantity, then sends an `ADD` request to the server.
   - Confirms the addition to the cart upon receiving a server response.
3. **CART**:
   - Sends a `CART` request to the server to retrieve and display cart contents.
4. **CHECKOUT**:
   - Sends a `CHECKOUT` request to finalize the purchase.
   - Displays a confirmation message or error if there is insufficient stock.

C. Example Client Code Structure

Here's a basic layout of client-side functions. Customize based on the project requirements:

```python
import socket

def view_products(client_socket):
    client_socket.send("VIEW".encode())
    response = client_socket.recv(1024).decode()
```

```python
    print("Available Products:")
    print(response)

def add_to_cart(client_socket, product_id, quantity):
    client_socket.send(f"ADD {product_id} {quantity}".encode())
    response = client_socket.recv(1024).decode()
    print(response)

def view_cart(client_socket):
    client_socket.send("CART".encode())
    response = client_socket.recv(1024).decode()
    print("Cart Contents:")
    print(response)

def checkout(client_socket):
    client_socket.send("CHECKOUT".encode())
    response = client_socket.recv(1024).decode()
    print("Checkout Confirmation:")
    print(response)

# Client setup
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 5000))

# Client menu loop
while True:
    print("1. View Products")
    print("2. Add to Cart")
    print("3. View Cart")
    print("4. Checkout")
    choice = input("Choose an option: ")

    if choice == '1':
        view_products(client_socket)
    elif choice == '2':
        product_id = input("Enter Product ID: ")
        quantity = input("Enter Quantity: ")
        add_to_cart(client_socket, product_id, quantity)
```

```
    elif choice == '3':
        view_cart(client_socket)
    elif choice == '4':
        checkout(client_socket)
        break
    else:
        print("Invalid choice. Try again.")

client_socket.close()
```

Step 4: Testing and Validation

1. **Run Server**:
   - Start the server program in one terminal.
2. **Run Client**:
   - Start the client program in a separate terminal.
   - Test each function to ensure the client-server communication works as expected.
3. **Debug and Refine**:
   - Test with various inputs and edge cases.
   - Ensure the inventory updates correctly after each transaction, and confirm that all transactions are logged properly.

Enhancements for Future Development

1. **Error Handling**:
   - Implement error handling for common issues (e.g., invalid product ID, insufficient stock).
2. **Database Integration**:
   - Move from `.txt` files to SQLite for more robust inventory and transaction management.
3. **GUI Implementation**:

- Add a graphical user interface using Tkinter to make the shopping experience more interactive.

4. **Multiple Client Handling**:

   - Expand the server to handle multiple client connections if needed.

5. **Real-Time Updates**:

   - Implement real-time inventory updates across clients, especially useful in environments with multiple shoppers.

This guide provides a structured approach for developing a simple yet fully functional client-server shopping system. Each step builds upon the previous, allowing you to implement and test functionality progressively. Adapt the specifics to suit your project's requirements and use this as a foundation for further enhancements, such as database integration and GUI.

In [ ]:

```
```